Milestone 2 Report

Bayesian Reasoning for LLM Agents under Uncertain and Conflicting Evidence

Student	Dhvanit Mahesuriya (AU2440039)
Programme	B.Tech Computer Science Engineering
Course	CSE 516, Probabilistic Graphical Models, Monsoon Semester 2026
Group / Track	g13 / Theoretical AI
Base paper	BayesAgent (Huang et al., AAAI 2026) [1]
Date	15th October 2026

1. Problem Statement

LLM agents often use extra information while answering a question. It can be a retrieved passage, an image caption or the output of a tool. This information is not always good. Some of it is wrong, some of it has nothing to do with the question, and sometimes two pieces disagree with each other. A good agent should become more sure when reliable evidence agrees with its answer, and less sure when reliable evidence disagrees. If the evidence is irrelevant, its confidence should stay about the same.

BayesAgent [1] adds hidden variables and Bayesian inference to LLM reasoning, and it gives better calibrated confidence on ScienceQA. But it does not study each piece of evidence separately, and it does not look at how the belief changes when evidence comes one piece at a time. My project is about this gap.

Research question: how can BayesAgent be extended to handle several pieces of evidence with different reliability, and how should the agent's belief and confidence change when new or conflicting evidence arrives? This is the same question as in M1. M2 is the stage where I build the technical base to answer it.

2. What I Did in M2

I started by reading the whole BayesAgent paper. Whenever a concept was new to me (the Dirichlet prior, the calibration loss, ECE, and how a verbalized PGM is different from a normal one) I watched videos on it until I could follow the derivation. Then I set up the official code and ran a small test on 3 questions to check that the pipeline works from start to end.

The GPU server I used has a driver that is too old for vLLM, which the official code needs. So I changed the inference part to use Hugging Face transformers with GPU and CPU offloading. This worked, but it made everything slow. The vPGM-N3 run on 50 questions took almost 20 hours, about 24 minutes per question. After that I ran CoT and Chameleon on the same 50 questions and computed accuracy and ECE for all three methods.

The other half of the work was on paper. I wrote down the graphical model for my extension, derived an update rule for the belief and proved its main properties. So at this point the base paper is reproduced on a small scale, and the extension exists as a formal model with maths but not as running code.

Done	Not done yet
vPGM-N3, CoT and Chameleon run on the same 50 questions. Accuracy, ECE and reliability diagrams computed.	BayesVPGM step (fitting λ) on the saved outputs.
Graphical model for the extension, with factorization and assumptions.	The extension as code, tested on the LLM.
Update rule for sequential evidence, with proofs and a worked example.	Measuring the confidence change ΔC under different kinds of evidence.

To keep things clear, the accuracy and ECE numbers in this report come from real runs. Everything about the extension is worked out on paper, and I say so wherever it matters.

3. The Graphical Model

3.1 The base model (vPGM-N3)

The base model has the inputs, a few hidden reasoning variables Z and the answer Y. The hidden variables are small judgements that the LLM has to make before it answers, for example whether the retrieved text is useful. The paper does not write probability tables for them. Each conditional probability is described in words, and the LLM gives a number for it while it answers. The answer probability is

$$P(Y \mid X) = \Sigma_Z \, P(Y \mid Q, Z) \cdot P(Z \mid X), \; P(Z \mid X) = \Pi_j \, P(Z_j \mid Pa(Z_j))$$

For every setting of Z we take its probability given the inputs, multiply by the probability of Y given that setting and the question, and add over all settings.

I used 3 hidden variables. The structure is fixed. It is the same for all 50 questions, and only the numbers change from question to question. The official code gives it as Q, C, M, I, K → Z_1; I → Z_2; I → Z_3; and Q, Z_1, Z_2, Z_3 → Y. I checked the diagram in Figure 1 edge by edge against this.

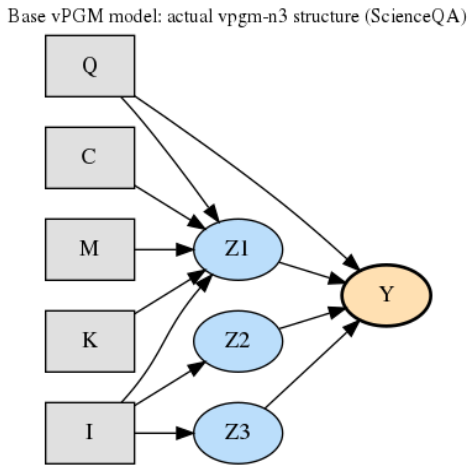


Figure 1. The vPGM-N3 structure used for every ScienceQA question.

Node	Meaning
Q	The question text.
C	Context, a short instruction such as 'select the better answer'.
M	Metadata: grade level, subject and topic.
I	Image caption and any text detected in the picture.
K	Knowledge retrieved from a search or knowledge source.
Z_1	Whether outside knowledge is needed to answer. It depends on all five inputs because the whole picture is needed to judge this.

Node	Meaning
Z_2	Whether the image caption is reliable. It depends only on I, since it is about the quality of the tool output.
Z_3	Whether the detected text is reliable. It also depends only on I.
Y	The final answer. It depends on Q and on Z_1, Z_2, Z_3.

An important point for my project is that this model already judges the quality of three fixed sources: the knowledge (Z_1), the caption (Z_2) and the detected text (Z_3). If the caption is poor, Z_2 is low and the confidence in the answer goes down. So some reliability judgement is already built in. What the base model does not do is handle any number of evidence items, let reliability depend on the question, follow the belief as evidence arrives in order, or give a numeric update rule whose behaviour can be proved. The extension below is about these four things.

3.2 My extension

In the extension every piece of evidence E_i gets its own node, and next to it a hidden reliability node R_i between 0 and 1. R_i says how much E_i should be trusted for this question, so it covers both relevance and trust. $R_i = 0$ means the evidence should be ignored. For ScienceQA, the evidence items are the retrieved knowledge (E_1), the caption (E_2) and the detected text (E_3). R_2 and R_3 play the same role as Z_2 and Z_3, with two differences: they also depend on the question, and there can be any number of items arriving over time.

The edges are as follows. X and E_i go into R_i, because the reliability of a passage depends on the question and on the passage. X, the evidence and the reliabilities go into the hidden variables Z_j, which is how reliability changes the belief. Earlier Z's can feed later Z's, as the LLM finds them. And Q and Z go into Y, as in the base model. The joint probability is

$$P(Y, Z, R \mid X, E_{1:m}) = [\ \Pi_i P(R_i \mid X, E_i)\ ] \cdot [\ \Pi_j P(Z_j \mid Pa(Z_j))\ ] \cdot P(Y \mid Q, Z)$$

To get the answer, Z and R are summed out. With no evidence (m = 0) this becomes the base vPGM, so BayesAgent is a special case of the model and I can use it as the 'without extension' comparison later. I chose this structure because reliability belongs to each source, so each source gets its own node. The model grows only linearly with the number of evidence items, and the hidden Z layer keeps what worked in the paper.

Assumptions. These are the assumptions I make, and where each one can break:

1. Y does not depend on the evidence or on R once Q and Z are known. Z should hold what the LLM learned from the evidence. This fails if the LLM reads the evidence directly and skips Z.
2. Each R_i is judged alone, given X and E_i. This keeps the model small. It fails if two sources just copy each other, because the same claim would be counted twice.
3. R_i does not depend on Y. Reliability should not be decided by the answer it supports, since that would be circular. In practice the LLM may still look at its own answer while rating the evidence.
4. The M sampled answers are treated as independent draws. This is needed for the Dirichlet step. It is only approximately true, because samples from the same prompt are similar, so the real sample size is below M.
5. Numbers stated by the LLM can be used as probabilities. It is the only way to get numbers without expert-made tables, but the numbers may be inconsistent or badly calibrated. The Dirichlet step is meant to correct some of this.

4. Learning

Structure learning and parameter learning are two different things here. Structure learning means finding the graph, that is, which hidden variables exist and which arrows connect them. In vPGM this is done by the LLM. It is given a prompt with the task, some input-output examples, the context and limits such as the maximum number of hidden variables, and it returns the variables and the arrows [1]. This avoids expert-made structures and expensive structure search. I used the official prompts with 3 hidden variables, which the paper finds to work best, so this part is done.

Parameter learning means finding the numbers. The vPGM itself has no trained parameters, because the LLM states its probabilities. The only trained numbers are in the BayesVPGM step: the prior strength λ and a loss weight β. In my extension the reliability weights r_i would be added to these. I worked this part out but have not fitted λ on my outputs yet.

The Dirichlet step works as follows. The LLM is queried M times, so I get M answers. I want to combine the votes for each answer with the probability that the vPGM gave to it. Let K be the number of choices and p_k the vPGM probability of choice k. The prior is Dirichlet(α) with $\alpha_k = \lambda\, p_k$. If choice k gets n_k votes, the posterior is again a Dirichlet, and its mean is

$$\pi_k = (n_k + \lambda\, p_k) \,/\, (M + \lambda\, \Sigma_j\, p_j)$$

This is the final predicted distribution. For a small λ it follows the votes, and for a large λ it follows the vPGM probabilities. To choose λ, the paper minimizes

$$L(\lambda) = L_c(\pi(\lambda)) + \beta \cdot L_v(\pi(\lambda)),\ L_v = (1/K)\ \Sigma_j\ |\ \bar{\pi}_j - \bar{y}_j\ |$$

with L-BFGS. L_c is cross-entropy. L_v compares the average predicted probability of each class ($\bar{\pi}_j$) with how often that class is correct ($\bar{y}_j$). It is like class-wise ECE without bins, so gradients can be used. Theorem 1 in the paper says that the best value of this loss gives perfect class-wise ECE, so the two parts of the loss do not work against each other. I use this result and do not re-prove it. Two practical points: the absolute value is not smooth at zero, so a sub-gradient is needed there, and fitting λ on the same 50 questions that I test on would look too good, so I plan to use cross-validation.

For the extension, the reliability of an item is $r_i = E[R_i \mid X, E_i]$, which comes from the LLM's own judgement of that item. It is used as a weight in the update in the next section. Since R is a node in the model, r_i could later be learned with the same calibration loss.

5. Inference

For a single question, the query is the answer distribution and the confidence, $P(Y \mid X)$ in the base model and $P(Y \mid X, E_{1:m})$ in the extension. The confidence is the probability given to the predicted answer. The method is prompting-based inference. One prompt makes the LLM go through the conditional probabilities step by step and state $P(Z_j \mid \text{parents})$ and $P(Y \mid Q, Z)$. This is repeated M times and the numbers are averaged, which approximates the sum over Z (Eq. 1 in [1]). Then the Dirichlet step above is applied. I use this method because the conditional probabilities exist only as sentences, so exact methods like variable elimination cannot be applied. The LLM does the sum over Z inside its reasoning, and the Dirichlet step on top is exact and cheap. With 3 hidden variables almost all the cost is LLM calls.

For evidence arriving one by one, the query is how the belief changes after each new item, $P_t(Y) \to P_{t+1}(Y \mid E_{t+1})$, and the confidence change ΔC. I keep a Dirichlet state α_t with total $A_t = \Sigma_k\, \alpha_{t,k}$, so the belief is $\pi_t = \alpha_t / A_t$. When a new item arrives, the LLM is asked how reliable it is (r, between 0 and 1) and which answer distribution it supports by itself ($\bar{s}$). Then

$$\alpha_{t+1} = \alpha_t + r \cdot \kappa \cdot \bar{s},\ \pi_{t+1} = \alpha_{t+1} \,/\, (A_t + r\, \kappa)$$

Here κ is a constant that says how many votes one fully trusted item is worth. A reliable item adds many votes, and an unreliable one adds almost none. I chose this update because it is conjugate, costs O(K) per step and needs only one LLM call per evidence item. Its behaviour can also be proved, which suits a theory project, and it fits my limited compute.

To measure the change, I fix a reference answer $\hat{y}_0$, which is the answer before any evidence. C_t is the probability given to $\hat{y}_0$ at step t, and $\Delta C = C_{t+1} - C_t$. With a fixed reference answer, a drop in confidence can be measured even if the top answer changes.

6. Mathematical Formulation and Checks

This section is worked out on paper. Let A be the total of α and $\pi = \alpha / A$ the current belief. Let $m = r\,\kappa$ be the amount of evidence added, and $\bar{s}$ the answer distribution the evidence supports.

Proposition 1. After the update, $\pi' = (1 - w)\,\pi + w\,\bar{s}$, where $w = m / (A + m)$.

Proof. $\pi'_k = (\alpha_k + m\,\bar{s}_k) / (A + m) = [A / (A+m)] \cdot (\alpha_k / A) + [m / (A+m)] \cdot \bar{s}_k$. ∎

So the new belief lies between the old belief and the evidence, and w decides how far it moves. Several things follow from this.

1. If there is no evidence, or if r = 0, then w = 0 and nothing changes, so ΔC = 0 exactly. Evidence that is judged irrelevant cannot move the belief.
2. $\Delta\pi_k = w\,(\bar{s}_k - \pi_k)$. The belief in answer k goes up if the evidence gives k more weight than the current belief, and goes down if it gives less. This is what 'supporting' and 'contradicting' evidence mean in this model.
3. $|\Delta\pi_k| \le w \le r\kappa / (A + r\kappa)$. One misleading item can move the belief only by a bounded amount, and the bound is smaller when r is low or when earlier evidence has already built up A.
4. w becomes smaller as A grows, so a belief that is already well supported moves less.
5. The final state is $\alpha_0 + \kappa \Sigma\, r_t\, \bar{s}_t$, which is the same in any order. This is good for consistency, but it is also a limitation, because the model cannot show order or recency effects.
6. If the same $\bar{s}$ keeps arriving with r above zero, A grows without limit and π moves towards $\bar{s}$.
7. Multiplying the pseudo-counts by r is the same as raising the likelihood to the power r, which is called a power posterior [9]. r = 1 is a fully trusted observation and r = 0 throws it away.

One design point came out of this. Irrelevance has to be shown by r close to 0, not by a uniform $\bar{s}$. If an irrelevant item had r above 0 and a uniform $\bar{s}$, Proposition 1 shows it would pull the belief towards uniform and lower the confidence, which is the wrong behaviour. This is the main reason I made reliability a separate node.

Here is a worked example that I checked by hand and then by calculation. Take K = 3, prior p = (0.6, 0.3, 0.1) and λ = 5, so α = (3, 1.5, 0.5). Vote counts n = (2, 1, 0) give posterior parameters (5, 2.5, 0.5) with A = 8, so π = (0.625, 0.3125, 0.0625). Now evidence supporting class 2 arrives with $\bar{s}$ = (0, 1, 0), r = 0.5 and κ = 1.

	Class 1	**Class 2**	**Class 3**
π before	0.6250	0.3125	0.0625
π after, from (5, 3, 0.5) / 8.5	0.5882	0.3529	0.0588
Change, computed directly	−0.0368	+0.0404	−0.0037
Change, from fact 2 above	−0.0368	+0.0404	−0.0037

Both ways give the same numbers. Evidence for class 2 lowers classes 1 and 3 and raises class 2, as fact 2 says, and setting r = 0 gives back the first row, as fact 1 says.

7. Benchmark and Baseline Setup

I tried to keep everything the same for the three methods except the method itself. All of them use the same 50 ScienceQA [4] questions (the 'test_50' subset), the same model (meta-llama/Meta-Llama-3-8B-Instruct, the model used in Table 1 of the paper), the same local server (Quadro RTX 5000, 16 GB, no external API), and the same metric script, eval/compute_metrics.py. Accuracy and ECE are computed with 10 equal-width bins, and ECE is shown $\times 10^2$ as in the paper.

The three methods are CoT with a stated confidence [2], Chameleon, which is a tool-using LLM with a stated confidence [6], and vPGM-N3 with 3 hidden variables and M = 3 samples. Each one uses its default settings from the official code. In Table 1 of the paper M = 1 for CoT and Chameleon and M = 3 for vPGM.

The only change to the original code is the backend. vLLM was replaced by Hugging Face transformers with GPU+CPU offloading, because of the old driver. The model, the dataset and the vPGM reasoning logic are the same as in the original repo. Part of the model runs on the CPU, which is why vPGM-N3 took 19 hours 56 minutes for 50 questions. The earlier 3-question runs were only a check of the pipeline, and I do not report them as results.

8. Initial KPI Results

Accuracy is the share of questions answered correctly. ECE (expected calibration error) shows how far the stated confidence is from the real accuracy: the answers are put into 10 confidence bins, and ECE = Σ_b (n_b/N) · | accuracy(b) − confidence(b) |. Lower is better.

Method	N	Correct	Accuracy	95% Wilson CI	ECE (×10²)
CoT (stated confidence)	50	43	86.00%	73.8 to 93.0	13.35
Chameleon	50	41	82.00%	69.2 to 90.2	11.19
vPGM-N3 (M = 3)	50	45	90.00%	78.6 to 95.7	9.79

Number correct = accuracy × 50. The intervals are Wilson 95% intervals from those counts.

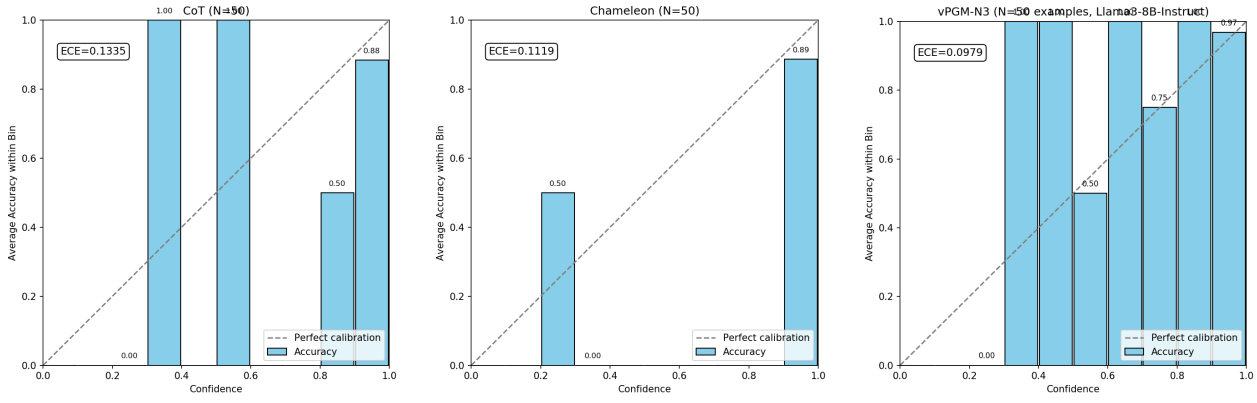


Figure 2. Reliability diagrams (10 bins, N = 50). Left to right: CoT (ECE 0.1335), Chameleon (ECE 0.1119), vPGM-N3 (ECE 0.0979). The number above each bar is the accuracy in that bin, and the dashed line is perfect calibration.

vPGM-N3 has the best accuracy and the lowest ECE of the three, which is the same direction as in the paper. But I cannot say from this that the methods really differ. The accuracy intervals overlap a lot, and vPGM beats CoT by only 2 questions out of 50. The ECE gaps are small enough to be noise at this sample size.

In the diagrams, most CoT and Chameleon answers fall in the top confidence bin (0.9 and above), where the real accuracy is only 0.88 and 0.89, so they are a little overconfident. vPGM-N3 spreads its confidence over more bins, which is what I would expect if the hidden variables adjust the confidence. But many of the middle bins hold only a few questions (their accuracy is 0, 0.5 or 1),

so this is only a rough picture.

My vPGM-N3 ECE of 9.79 is far from the 1.67 in the paper. I am not claiming to have reproduced that number. I used 50 questions, where ECE is noisy and tends to come out too high, and I have not run the BayesVPGM Dirichlet step yet, which is what gives the paper its lowest ECE. What the run does show is that the whole vPGM pipeline works in my setup and that the KPIs can be computed.

The confidence change ΔC is defined but not measured yet. From the maths of the update rule I expect four things, and I write them down now so that I cannot change them after seeing results. Reliable supporting evidence should give ΔC above 0. Reliable contradicting evidence should give ΔC below 0. Irrelevant evidence should give ΔC close to 0, within the normal run-to-run noise. And for misleading evidence, the reliability-aware update should move the belief less than an update that ignores reliability. Whether these hold when r and $\bar{s}$ come from a real LLM is the main question for M3. The paper also reports mean $P(Z_2)$ of 0.86 on clean and 0.42 on noisy A-OKVQA data [1], and I plan a similar mismatch check on my own evidence variants.

9. Changes Since M1

Several things changed once I started building. The biggest is scale. With about 24 minutes per question, I cannot run the full benchmark, so the work is limited to 50 questions, and the LLM experiments with evidence will use about 20 to 30 questions. The backend also changed from vLLM to transformers, as described above.

M1 planned to compare direct prompting, CoT, BayesAgent and BayesVPGM. In M2 I report CoT, Chameleon and vPGM-N3. The BayesVPGM step will be run offline on the saved outputs in M3. vPGM-N2, vPGM-N4 and Chameleon+ were started but did not finish in time, so they are not reported. In M1 I said the reliability representation would be refined. It is now fixed: reliability is a hidden node for each evidence item, and it works as a weight on the Dirichlet pseudo-counts. Only ScienceQA is used in M2. A-OKVQA clean and noisy is the natural next dataset for mismatch detection. I leave out ChatCoach, since the paper itself does not apply BayesVPGM there.

On the position against existing work, BayesAgent [1] is still the closest paper. The paper describes BIRD [8] as a Bayesian wrapper for LLMs that works only for binary decisions, so it does not cover multi-class questions like these. The calibration loss in [1] is inspired by class-wise ECE and Dirichlet calibration [7], and verbalized confidence comes from [5], which I use for the baselines. Looking at the real vPGM-N3 structure made my claim narrower than I first thought. The base model already has Z_2 and Z_3, which judge the caption and the detected text. So the new part of my work is not the idea of judging source quality. It is the general form (any number of evidence items, with reliability that depends on the question), the sequential update with proved properties, and the ΔC measurement. This has to be shown by experiments in M3.

10. Limitations

The main limit is the sample size. With 50 questions the accuracy has an interval of about ±8 to 9 points, and ECE is noisy, so all comparisons here are preliminary. vPGM also uses 3 samples while the baselines use 1, so part of the better calibration may come from the extra samples and not from the model structure. I need a sample-matched comparison for that. I also used one model and one run for each method, so I have no variation over seeds.

On the modelling side, the LLM-stated probabilities may be inconsistent, and the sampled answers are not truly independent. The extension has not been tested with a real LLM, so the properties are proved only for the update rule when r and $\bar{s}$ are given. The update is also order independent, so it cannot show recency effects. Finally, I did not compare the transformers backend with vLLM directly, so I cannot rule out small differences in the outputs.

11. Reproducibility

The M2 folder follows the course layout: README.md, Report/, Video/, Code/src/ with its own README.md, Data/README.md and Results/. The code is the official repository of [1] with my transformers-backend change. Seeds, library versions and result file names are in Code/README.md. The metrics and diagrams were made with these commands, run from the repository root:

```
python3 eval/compute_metrics.py --results_file
results/my_scienceqa/test_50/cot_Meta-Llama-3-8B-Instruct_run-0.json --title "CoT" --output
results/reliability_cot_50.png
python3 eval/compute_metrics.py --results_file
results/my_scienceqa/test_50/chameleon_Meta-Llama-3-8B-Instruct_run-0.json --title "Chameleon" --output
results/reliability_chameleon_50.png
```

The console printed: CoT, N = 50, accuracy 86.00%, ECE 13.3520 ($\times 10^2$ = 13.35). Chameleon, N = 50, accuracy 82.00%, ECE 11.1853 ($\times 10^2$ = 11.19). The vPGM-N3 run gave accuracy 90.00% and ECE 9.79 ($\times 10^2$). Each saved JSON file has the problem id, the correct answer, the full LLM response for each sample (with the stated $P(Z_1)$, $P(Z_2)$, $P(Z_3)$) and the averaged answer distribution.

12. Timeline and Next Steps

Phase	Planned work	Status
Phase 1 (Weeks 1-3)	Understand LLM uncertainty, define the problem, study BayesAgent and recent work.	Completed
Phase 2 (Weeks 4-6)	Reproduce BayesAgent and implement baselines.	Mostly done. BayesVPGM step pending.
Phase 3 (Weeks 7-9)	Multiple evidence and controlled evidence conditions.	Started. Model and update rule written.
Phase 4 (Weeks 10-12)	Sequential evidence and Bayesian belief updates.	Planned
Phase 5 (Weeks 13-14)	Ablation, calibration analysis and final comparison.	Planned
Phase 6 (Final)	Final paper and presentation.	Planned

The next steps, in order, are these. The first three need no new LLM calls.

1. Run BayesVPGM offline on the saved vPGM outputs: read the per-sample labels and probabilities, fit λ with cross-validation, and report ECE with a bootstrap interval.
2. Compare the hidden variables: average $P(Z_1)$, $P(Z_2)$, $P(Z_3)$ for correct and wrong answers, and check how they correlate with correctness.
3. Write the reliability-weighted update as code, with tests for the facts in Section 6 and for the worked example.
4. Build evidence variants (reliable, irrelevant, misleading, conflicting) for 20 to 30 questions, measure ΔC for each type and test the four expectations.
5. Run ablations (no reliability node, an update that ignores reliability, N = 2 and 4), sample-matched baselines and Chameleon+.
6. Run A-OKVQA clean and noisy for mismatch detection, with more questions and repeated runs.

13. Conclusion

The first milestone set the research direction, and M2 built the technical base. The vPGM pipeline now runs on Llama-3-8B-Instruct, and CoT and Chameleon were run under the same conditions on the same 50 questions. vPGM-N3 gave the best accuracy (90%) and the lowest ECE (9.79), but with 50 questions the differences are not statistically clear. The graphical model with evidence and reliability nodes is written down, and a reliability-weighted sequential update is derived with proofs of its main properties. The next milestone will fit the BayesVPGM step, code the update rule, and measure the confidence change on the LLM for reliable, irrelevant, misleading and conflicting evidence.

References

[1] H. Huang, X. Shen, G.-Y. Hao, S. Wang, L. Meng, D. Liu, D. A. Duchene, H. Wang, S. Bhatt. BayesAgent: Bayesian Agentic Reasoning Under Uncertainty via Verbalized Probabilistic Graphical Modeling. AAAI, 2026. arXiv:2406.05516.

[2] J. Wei et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS, 2022.

[3] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger. On Calibration of Modern Neural Networks. ICML, 2017.

[4] P. Lu et al. Learn to Explain: Multimodal Reasoning via Thought Chains for Science Question Answering. NeurIPS, 2022.

[5] M. Xiong et al. Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs. arXiv:2306.13063, 2023.

[6] P. Lu et al. Chameleon: Plug-and-Play Compositional Reasoning with Large Language Models. arXiv:2304.09842, 2023.

[7] M. Kull et al. Beyond Temperature Scaling: Obtaining Well-Calibrated Multi-Class Probabilities with Dirichlet Calibration. NeurIPS, 2019.

[8] Y. Feng, B. Zhou, W. Lin, D. Roth. BIRD: A Trustworthy Bayesian Inference Framework for Large Language Models. ICLR, 2025.

[9] N. Friel, A. N. Pettitt. Marginal likelihood estimation via power posteriors. J. R. Stat. Soc. B, 70(3):589-607, 2008.